

Architecting a High-Fidelity London Stock Exchange Simulation: A Technical Blueprint

Introduction

The development of financial market simulations presents a fundamental choice between two distinct paradigms. The first, rooted in financial mathematics, models asset price movements as a continuous-time stochastic process, often using frameworks like Geometric Brownian Motion (GBM) to generate probabilistic price paths. While valuable for certain types of risk analysis and option pricing, this approach fundamentally abstracts away the very mechanisms that create the price. The second paradigm, demanded by the study of market microstructure, treats the exchange not as a source of random variables but as a complex, deterministic, rule-based system that processes discrete events—orders, cancellations, and trades—to produce market outcomes. A high-fidelity trading simulation, capable of accurately testing the behavior of sophisticated algorithmic strategies, must adhere to this latter, more rigorous model.

This report provides a definitive technical blueprint for constructing a high-fidelity Python simulation of the London Stock Exchange (LSE). The objective is to move beyond simplistic backtesters and create a digital twin of the exchange's core operational logic. This requires a deep and nuanced understanding of the LSE's market microstructure, from its official rulebooks to its specific technological implementations. The scope of this analysis will encompass the complete lifecycle of a trading day, the full spectrum of supported order types and their validity conditions, the intricate mechanics of the matching engine, and the critical differences between the LSE's various trading services.

The London Stock Exchange serves as a particularly compelling subject for such a simulation. As a major international financial hub, it operates not as a monolithic entity but as a collection of sophisticated, distinct market models tailored to securities with varying liquidity profiles. The contrast between the fully electronic, order-driven Stock Exchange Electronic Trading Service (SETS) for blue-chip stocks and the hybrid auction-based SETSx for less liquid securities presents a significant but essential modeling challenge. Successfully simulating the LSE, therefore, requires an architecture that is not only fast and accurate but also flexible enough to accommodate these diverse and complex rule sets. This report will deconstruct these rules, analyze the shortcomings of existing open-source tools, and present a clear, phased roadmap for building a simulation that captures the true operational reality of a world-class exchange.

Section 1: Deconstructing the London Stock Exchange's Market Microstructure

To build an accurate model of any complex system, one must first understand its foundational principles and governing documents. The London Stock Exchange's operations are not defined by a single manual but by a hierarchical and interconnected set of rules and technical specifications. A developer aiming for high fidelity must synthesize information from these

sources to form a complete picture of the market's mechanics.

1.1 The Documentary Framework: A Tripartite System of Rules

The LSE's operational logic is codified across three primary categories of documentation, each serving a distinct purpose. An effective simulation architecture must reflect this separation of concerns.

- **The Legal Layer - *Rules of the London Stock Exchange*:** This core document establishes the high-level, legally binding framework for all member firms. It defines key terms (often in bold text), outlines categories of membership, and sets the general principles of conduct, including prohibitions on misleading acts and price manipulation. For a simulator developer, this document provides the foundational context—the "why" behind the technical rules. For example, it makes the crucial distinction between a member firm acting as an agent versus a principal, which has implications for trade reporting, and defines what constitutes an "on Exchange" trade, which is the scope of the simulation. While much of its content relates to compliance and legal obligations not directly modeled in a matching engine, understanding its structure is essential for interpreting the more technical documents.
- **The Functional Layer - *Guide to the Trading System (MIT201)*:** This is the most critical document for building the simulation's logic engine. The *Guide* provides a detailed functional overview of the LSE's trading platform, Millennium Exchange, explaining the "how" of the entire trading process. It meticulously describes the sequence of a trading day, the behavior of every supported order type and time-in-force (TIF) condition, the mechanics of auctions, and the logic for market control events like trading halts. This document is the primary source for designing the simulator's state machine, order validation logic, and auction uncrossing algorithm.
- **The Parameter Layer - *Millennium Exchange & TRADEcho Business Parameters*:** This document, typically distributed as an XLSX spreadsheet, contains the specific, configurable values that govern the live trading system. It is the "what" that fine-tunes the "how" described in the *Guide*. It includes crucial data points such as auction timings for different market segments, price monitoring percentages, tick size tables (price format schedules), maximum order values, and minimum size thresholds for complex orders like Icebergs and Hidden orders. A high-fidelity simulator must be architected to load and utilize these parameters dynamically for each security, rather than hardcoding them. This ensures the simulation can accurately model different instruments and adapt to rule changes published by the exchange.

The interplay of these three documents is fundamental. The *Rules* set the legal stage, the *Guide* describes the actors and the plot, and the *Parameters* document provides the specific stage directions for each performance. A naive implementation that conflates these concerns will be brittle and inaccurate. A sophisticated architecture will mirror this separation, with a configuration loader for the *Parameters*, a core state machine and rule engine driven by the *Guide*, and a contextual understanding informed by the *Rules*.

1.2 Core Trading Services: A Spectrum of Liquidity

The LSE is not a single market but a collection of services, each designed to provide an appropriate trading environment for securities with different liquidity characteristics. A comprehensive simulator must be capable of modeling these distinct environments.

- **SETS (Stock Exchange Electronic Trading Service):** This is the LSE's flagship electronic order book, designed for the most liquid securities, including all constituents of the FTSE All-Share Index, highly traded AIM securities, and various other instruments. It is a pure, order-driven market operating on a continuous basis throughout the trading day, bookended by opening and closing auctions. Its core is a Central Limit Order Book (CLOB) that matches orders based on price-time priority. This is the primary model that most of this report will focus on, as it represents the most complex, high-volume trading environment on the LSE.
- **SETSqx (Stock Exchange Electronic Trading Service – quotes and crosses):** This is a specialized, hybrid service designed for securities that are less liquid than those on SETS. Modeling SETSqx correctly is a significant step up in complexity because it does not feature continuous trading. Instead, it operates on a series of scheduled auctions throughout the day, for example at 8:00, 9:00, 11:00, 14:00, and 16:35. The simulator must be able to handle two distinct sub-models of SETSqx :
 1. **SETSqx with Market Makers:** This model combines the periodic electronic auctions with a parallel, non-electronic, quote-driven market-making facility. Registered market makers are obliged to provide firm, two-way quotes during trading hours.
 2. **SETSqx with NO Market Makers:** This is a pure periodic auction model. Following each auction uncrossing, the order book immediately enters the call period for the next auction, and orders are visible upon submission.
- **SEAQ (Stock Exchange Automated Quotation system):** This is a purely quote-driven market maker system, primarily for AIM securities and fixed-interest securities not traded on SETS or SETSqx. In this model, brokers do not trade against an anonymous order book but directly with market makers who display bid and offer prices. While less relevant for a developer focused on building a CLOB simulator, its existence underscores the LSE's multi-faceted approach to market structure.

1.3 The Anatomy of a Trading Day (SETS Model)

The standard trading day on the SETS platform is a structured sequence of distinct phases, each with its own specific rules governing permissible actions. A simulator's core logic must be built around a state machine that accurately transitions through these phases, driven by a simulated clock. The typical schedule, in London time, is as follows :

- **Trade Reporting (07:15 – 07:50):** Off-book trades can be reported to the exchange, but the electronic order book is not yet open for order entry.
- **Opening Auction Call (07:50 – 08:00):** The order book opens for order entry, modification, and cancellation. No matching occurs during this phase. The system disseminates indicative auction prices and volumes to provide transparency.
- **Continuous Trading (08:00 – 16:30):** The opening auction "uncrosses" at 08:00, and the market transitions to the main trading phase. Orders are matched continuously based on price-time priority as they arrive. This is the longest phase of the trading day.
- **Closing Auction Call (16:30 – 16:35):** Continuous trading ceases, and the market enters the closing auction call. As with the opening auction, orders can be entered, modified, and cancelled, but no matching occurs. This phase determines the official closing price for many securities.
- **Closing Price Crossing Session (CPX):** For certain securities, a short, modified trading session may follow the closing auction. During CPX, trading can only occur at the single

price determined by the closing auction.

- **Order Maintenance & Post-Close (16:35 – 17:15):** After the close, a period exists for order maintenance (cancellation) and further off-book trade reporting.

This schedule is subject to variation, such as early closes on certain holidays. Furthermore, the system must be able to handle unscheduled interruptions, such as a **Volatility Auction** triggered by a price monitoring breach during continuous trading, or a **Trading Halt** invoked by the exchange.

Table 1.1: LSE Trading Day Phases (SETS Model)				
Phase Name	Standard Timings (UTC)	Key Purpose	Permissible Order Actions	Matching Activity
Opening Auction Call	07:50 - 08:00	Price discovery for the market open.	Entry, modification, and cancellation of orders.	None. Indicative price/volume disseminated.
Continuous Trading	08:00 - 16:30	Main trading session for continuous order matching.	Entry, modification, and cancellation of orders.	Continuous matching based on Price-Time Priority.
Closing Auction Call	16:30 - 16:35	Price discovery for the official closing price.	Entry, modification, and cancellation of orders.	None. Indicative price/volume disseminated.
Closing Price Crossing Session (CPX)	Follows Closing Auction	Further trading opportunity at the closing price.	Entry and cancellation of orders at the closing price only.	Continuous matching at the closing auction price only.
Post-Close	16:35 - 17:15	Housekeeping and final reporting.	Order cancellation, off-book trade reporting.	None.

1.4 The Central Limit Order Book (CLOB) and Matching Priority

The heart of the SETS trading service is the Central Limit Order Book (CLOB), which matches buy and sell orders according to a strict set of rules.

- **Core Algorithm: Price-Time Priority:** The fundamental matching logic is **Price-Time Priority**, also known as First-In-First-Out (FIFO). This two-tiered system works as follows:
 1. **Price Priority:** Orders with more aggressive prices have priority. For buy orders, a higher price is more aggressive. For sell orders, a lower price is more aggressive.
 2. **Time Priority:** For all orders at the same price level, the one that was entered into the book first has priority. An incoming marketable order will execute against resting orders in the book according to this priority until the incoming order is filled, or it exhausts all available liquidity at its limit price.
- **Handling of Complex Orders:** The simple price-time logic is extended with specific rules for more complex order types, which a high-fidelity simulator must implement correctly.
 - **Iceberg Orders:** These orders display only a portion (the "peak") of their total size. When the visible peak is fully executed, it is refreshed. A critical and often overlooked detail is how incoming volume is allocated against multiple resting

Iceberg orders at the same price. After the visible peaks are executed in time priority, any remaining incoming volume is allocated to the hidden portions of the Iceberg orders on a **pro-rata basis**, not by time priority.

- **Hidden Orders:** Fully non-displayed orders (which must meet a Large-in-Scale 'LIS' threshold) have lower priority than any visible order at the same price, including the visible peak of an Iceberg order.
- **Minimum Quantity at Touch (MQAT):** For certain, typically less liquid, non-FTSE350 securities, the LSE implements an MQAT rule. This rule states that an incoming passive limit order (one that does not immediately trade) must be at least a certain percentage of the security's Normal Market Size (NMS) to be allowed to set a new Best Bid or Offer (BBO). Orders that fail this check are rejected. This is a crucial anti-flicker mechanism designed to improve market quality, and its omission would be a significant oversimplification.

Section 2: The LSE Rulebook in Code: A Blueprint for Implementation

Translating the LSE's comprehensive rulebook into a functioning software simulation is a formidable task. It requires moving beyond high-level concepts to concrete architectural decisions and algorithmic logic. This section provides a practical blueprint for this translation process, focusing on the core components required for a high-fidelity model.

2.1 Modeling the Trading Day Lifecycle: A Finite-State Machine Approach

The most robust and accurate way to model the LSE's structured trading day is by implementing a **Finite-State Machine (FSM)**. The core of the simulator's event loop will not be a simple order processor but a state manager that dictates which actions are permissible at any given moment.

- **States and Transitions:** Each distinct trading phase identified in Section 1.3 should be a state in the FSM (e.g., PRE_TRADE, OPENING_AUCTION_CALL, CONTINUOUS_TRADING, VOLATILITY_AUCTION, CLOSING_AUCTION_CALL, CPX, POST_CLOSE). Transitions between these states are driven by time-based events from a simulated clock. For example, at 08:00:00, the FSM transitions from OPENING_AUCTION_CALL to CONTINUOUS_TRADING. This transition triggers the auction uncrossing logic.
- **State-Dependent Logic:** Each state must define the valid set of operations. When an order arrives, the FSM first checks its current state. If the state is CONTINUOUS_TRADING, the order is passed to the matching engine. If the state is OPENING_AUCTION_CALL, the order is added to the book but not matched. If the state is POST_CLOSE, the order would be rejected. This architecture elegantly enforces the rules described in the *Guide to the Trading System*.
- **Parameterization:** The transition times for the FSM should not be hardcoded. They must be loaded from the *Millennium Exchange & TRADEcho Business Parameters* document for the specific market segment being simulated. This allows the same FSM logic to handle different trading schedules, such as those for SETSqx, which has multiple intra-day auctions.

2.2 Implementing LSE Order Types and Validity Conditions

A significant portion of the simulation's complexity lies in correctly modeling the full spectrum of LSE order types and their intricate validity rules. This goes far beyond basic Market and Limit orders and requires a system that can handle the complex interplay between order type, Time-in-Force (TIF), and the current market state.

The simulator must implement classes for each major order type, including but not limited to: Limit, Market, Stop, Stop Limit, Iceberg, Hidden (including Mid-Price Pegged), and Executable Quotes. The processing logic for an incoming order must then perform a rigorous validation check based on the rules detailed in the *Guide to the Trading System (MIT201)*.

For example, the rules dictate specific behaviors that must be coded:

- An **Immediate or Cancel (IOC)** or **Fill or Kill (FOK)** order is rejected if submitted during any auction call phase.
- A **Stop Limit Order** can be submitted during an auction, but it is "parked" and only becomes eligible for election (i.e., entry into the book as a standard Limit Order) when continuous trading begins.
- A **Mid-Price Pegged Order** is also parked during an auction. Upon returning to continuous trading, it is re-validated against the minimum size requirements and may be expired if conditions have changed.

The following table synthesizes the complex validity rules from the LSE's technical documentation into a single, developer-focused matrix. It represents the core logic that the simulator's order validation engine must implement.

Table 2.1: LSE Order Type and Time-in-Force Validity Matrix (SETS Model)			
Order Type	Time-in-Force (TIF)	Auction Call Phase	Regular Trading Phase
		(Opening, Closing, Volatility)	
Limit	DAY, GTD, GTT, ATC, GFA, GFS	Accept (Added to book, no match)	Accept (Matches or rests on book)
	IOC, FOK	Reject	Accept (Immediate execution attempt)
Market	OPG, ATC, GFA	Accept (Added to book, no match)	N/A (Market orders become IOC/FOK)
	DAY, IOC, FOK	Reject	Accept (Immediate execution attempt)
Iceberg	DAY, GTD, GTT, ATC, GFA, GFS	Accept (Added to book, no match)	Accept (Matches or rests on book)
	IOC, FOK	Reject	Accept (Immediate execution attempt)
Hidden (inc. Mid-Price Pegged)	DAY, GTD, GTT	Park (Becomes inactive until Regular Trading)	Accept (Rests on book, matches dark)
	IOC, FOK	Reject	Accept (Immediate

Table 2.1: LSE Order Type and Time-in-Force Validity Matrix (SETS Model)			
			execution attempt)
Stop / Stop Limit	DAY, GTD, GTT	Park (Eligible for election at start of Regular Trading)	Accept (Parked until stop price is triggered)
	All Others	Reject	Reject

Note: This table is a high-level summary. The full implementation must account for all TIFs (e.g., GFX, CPX) and the specific behaviors for SETSqx segments as detailed in the MIT201 guide.

2.3 The Auction Uncrossing Algorithm: A Multi-Stage Priority Logic

The auction uncrossing process is not a simple price match; it is a sophisticated algorithm designed to maximize liquidity and establish a fair, representative price. A simulator must implement this multi-stage logic precisely as specified in the *Guide to the Trading System*. The logical flow for determining the uncrossing price is as follows:

1. **Principle 1: Maximize Executable Volume.** The system calculates the total number of shares that could trade at every possible price level within the order book. The price (or prices) that allows the maximum number of shares to be executed is selected.
2. **Principle 2: Minimize Surplus.** If multiple prices satisfy Principle 1, the system chooses the price that leaves the smallest remaining imbalance (the number of unmatched shares on the buy or sell side).
3. **Principle 3: Apply Market Pressure.** If a tie still exists after Principle 2, the market pressure rule is applied. If the imbalance is on the buy side (more buy orders than sell orders), the highest price is chosen. If the imbalance is on the sell side, the lowest price is chosen. This ensures the price moves in the direction of the excess demand.
4. **Principle 4: Reference Price Tie-Breaker.** If the rules above still result in a tie (e.g., a zero imbalance at multiple price points), the uncrossing price is the one closest to a pre-defined reference price, typically the last automated trade price or a previous closing price.

This core algorithm must be wrapped with logic to handle **auction extensions**, which are critical for market stability:

- **Price Monitoring Extension:** If the indicative uncrossing price at the end of the call period deviates from a reference price by more than a configured percentage (defined in the *Business Parameters*), the auction call is automatically extended for a short period (e.g., 5 minutes) to allow participants to react.
- **Market Order Extension:** If unpriced Market Orders would remain unexecuted at the indicative price, the auction is also extended to attract more liquidity and prevent these orders from being cancelled.
- **Randomization:** To prevent participants from "gaming" the exact moment of the close, the final uncrossing is preceded by a short, randomized period (e.g., 0-30 seconds). The simulator must include this random delay to be truly realistic.

2.4 Incorporating Essential Business Parameters

A high-fidelity simulator cannot have its core rules hardcoded. It must be designed to be parameter-driven, allowing it to accurately model any security on the LSE by loading the correct configuration. A dedicated configuration module should parse the *Millennium Exchange & TRADEcho Business Parameters* document and make these values available to the relevant parts of the simulation engine.

Key parameters that must be configurable include:

- **Auction Timings:** The start and end times for all scheduled auctions, which vary significantly between SETS and SETSx segments.
- **Price Monitoring Percentages:** The tolerance levels for triggering price monitoring extensions in auctions and volatility halts in continuous trading.
- **Tick Sizes:** The minimum valid price increments for each security, which are often dependent on the security's price level. The simulator must enforce these tick sizes on all incoming orders.
- **Order Size and Value Limits:** Maximum order values and minimum sizes for orders like Icebergs and Hidden orders.
- **Large-in-Scale (LIS) Thresholds:** The minimum size required for an order to be designated as fully hidden, a key requirement under MiFID II regulations.

By externalizing these rules into a configuration file, the simulator becomes a flexible and powerful tool that can be easily updated as the exchange modifies its rules, and can accurately model the specific trading conditions of thousands of different instruments.

Section 3: Analysis of Open-Source Trading Simulators

The open-source landscape offers a wide array of tools related to trading, but a critical analysis reveals a significant gap between what is available and what is required to build a high-fidelity LSE simulation. Existing projects generally fall into categories that solve related, but distinct, problems, and nearly all make significant architectural oversimplifications.

3.1 A Taxonomy of Simulators

To understand the state of open-source tools, it is useful to categorize them by their primary purpose and architectural focus.

- **Strategy Backtesters:** This is the most common category. Projects like Backtrader , Zipline , and the crypto-focused freqtrade are designed to test trading *strategies*. Their core function is to provide an API for processing historical data (usually OHLC bars), calculating technical indicators, and managing a portfolio. The actual trade execution is heavily abstracted; they do not simulate a limit order book or a matching engine in detail. They are fundamentally unsuitable for microstructure simulation.
- **Limit Order Book (LOB) Implementations:** These are more specialized projects that focus on creating an efficient data structure and matching logic for a continuous limit order book. Examples include PyLimitBook , lobby , and limit-order-book. These projects often provide a fast and correct implementation of price-time priority matching. They can serve as an excellent starting point for the core matching component (Phase 1 of the roadmap), but they are not complete simulators, as they lack the crucial state management for different trading phases.
- **Matching Engine Simulators:** A more advanced category of projects that aim to simulate

an exchange server. PythonMatchingEngine is a standout example, designed for replaying historical Level 3 (full order book) data and even simulating network latency. The limit-order-book-market-making project provides a client-server architecture to simulate a market maker interacting with an exchange. These projects are much closer to the user's goal but, as will be detailed, still have critical limitations.

- **Full-Fledged Trading Platforms:** The most sophisticated open-source projects are NautilusTrader and QuantConnect/LEAN. These are production-grade, event-driven platforms with extensive features, nanosecond resolution, and support for multiple asset classes. Their primary architectural purpose, however, is to be a *client* that connects to various exchanges, not to *be* the exchange itself. They are designed to react to market data events, not to generate those events according to an internal set of exchange rules.

3.2 Common Architectural Oversimplifications

When measured against the detailed LSE rulebook, nearly all open-source simulators exhibit significant oversimplifications. These are not minor omissions but fundamental architectural shortcomings that prevent them from being used for high-fidelity modeling out of the box.

- **The Single-State Fallacy:** The most pervasive oversimplification is the assumption that the market is always in a state of continuous trading. The complex, multi-phase nature of the trading day is ignored. Crucial price-discovery mechanisms like opening, closing, and volatility auctions are completely absent from projects like PythonMatchingEngine and limit-order-book-market-making. This is the single biggest gap between these tools and reality.
- **Incomplete Order Type and TIF Support:** Most simulators implement only basic Market and Limit orders with DAY or GTC time-in-force conditions. The rich and complex ecosystem of LSE order types—Iceberg, Hidden, Pegged, Stop—and their highly specific, state-dependent behaviors are not modeled. The intricate matrix of validity based on TIFs like At The Close (ATC), Good For Auction (GFA), and Good For Scheduled Auction (GFS) is a level of detail that is almost universally omitted.
- **Hardcoded Parameters:** Many simpler simulators hardcode critical business parameters like tick sizes or lot sizes. This makes them inflexible and inaccurate, as these parameters vary by security and are subject to change by the exchange. A high-fidelity model must be parameter-driven, loading these values from an external source as the real exchange does.
- **Lack of Regulatory and Microstructure Nuances:** Finer-grained rules that have a real impact on market dynamics are typically not implemented. This includes MiFID II-driven requirements like Large-in-Scale (LIS) thresholds for hidden orders and exchange-specific mechanisms like Minimum Quantity at Touch (MQAT) designed to improve market quality.

3.3 In-Depth Review: PythonMatchingEngine vs. NautilusTrader

A deeper analysis of the two most promising candidates for adaptation reveals the specific engineering challenges involved.

- **PythonMatchingEngine :**
 - **Strengths:** Its key advantage is its purpose-built design as a matching engine simulator. It is highly performant (capable of processing ~150k orders/second), is designed to replay historical L3 data tick-by-tick, and includes a model for network latency. This makes it a very strong candidate for implementing the core LOB and

matching logic of the continuous trading phase.

- **Weaknesses and Adaptation Challenge:** Its primary weakness is its single-state architecture. It is fundamentally a continuous matching engine and has no built-in concept of auctions, market halts, or different trading sessions. To adapt it for a full LSE simulation, one would need to build the entire trading day state machine, the auction uncrossing algorithm, and the complex order validity logic *around* its core order book. This would involve significant new development, treating the existing engine as a component rather than a complete framework.
- **NautilusTrader :**
 - **Strengths:** This is a production-grade, professionally engineered platform. Its event-driven architecture, written in a high-performance Rust/Python stack, is extremely robust. It supports nanosecond timestamp resolution, a wide array of advanced order types (including AT_THE_OPEN and AT_THE_CLOSE TIFs), and a modular adapter system for connecting to different venues. Its core design principles of correctness and performance are well-aligned with the goal.
 - **Weaknesses and Adaptation Challenge:** Its fundamental design is that of a *trading platform* (a client), not an *exchange simulator* (a server). It is built to receive market data and send orders, not to be the source of that data by enforcing rules on a collective of participants. The challenge in adapting NautilusTrader is therefore architectural. It would require a conceptual "inversion" of its logic. A developer would need to build a custom "LSE Venue Simulator" adapter. This adapter would not connect to an external feed; it would *be* the feed. It would need to contain the LSE state machine, the LOB, the auction logic, and the matching engine. It would receive order requests from strategies running on the platform and, instead of sending them to a real exchange, would process them internally according to the LSE rules, publishing the resulting market data (trades, book updates) back onto the platform's internal message bus. This is a non-trivial but potentially powerful approach, as it would leverage the platform's robust infrastructure.

No existing open-source project provides a turnkey solution. The developer must choose between building the complex state-management and event-driven architecture from scratch (starting with a simpler LOB library) or undertaking a significant re-architecting of a sophisticated platform like NautilusTrader to repurpose its components for simulation.

Section 4: A Phased Roadmap for Building a High-Fidelity LSE Simulator

Given the complexity of the task and the lack of a complete open-source solution, a phased, incremental development approach is critical for managing risk and ensuring a robust final product. This roadmap breaks down the project into logical, achievable stages, starting with a basic core and progressively adding layers of realism and complexity.

4.1 Phase 1: The Core - Continuous Trading Engine

The initial phase focuses on building the foundational component of the simulator: a functioning Central Limit Order Book (CLOB) for the continuous trading phase. This creates a tangible, testable baseline.

- **Objective:** Implement a single-state, continuous trading market for a single security with

basic order types.

- **Tasks:**

1. **Develop the Limit Order Book (LOB) Class:** Create a performant LOB data structure. This will involve managing separate data structures (e.g., sorted dictionaries or trees) for the bid and ask sides. The primary matching algorithm will be strict Price-Time Priority. Projects like `PythonMatchingEngine` can provide inspiration for efficient implementation.
2. **Implement Basic Order Types:** Create classes for `LimitOrder` and `MarketOrder`.
3. **Implement Basic Time-in-Force (TIF):** Implement the logic for DAY (expires at end of simulation), IOC (Immediate or Cancel), and FOK (Fill or Kill) orders.
4. **Create a Simple Event Loop:** Build a basic loop that can process a stream of orders from a file or queue, pass them to the LOB for matching, and record the resulting trades.
5. **Enforce Tick Sizes:** Implement a simple mechanism to validate incoming order prices against a valid tick size schedule, rejecting non-compliant orders.

At the end of this phase, the developer will have a working, albeit simplified, matching engine that correctly models the LSE's continuous market in isolation.

4.2 Phase 2: The Brain - Trading Day State Machine

This phase represents the most significant leap in complexity and is the key differentiator between a simple matching engine and a true exchange simulator.

- **Objective:** Introduce the concept of distinct trading phases and the logic for auctions.

- **Tasks:**

1. **Implement the Finite-State Machine (FSM):** Refactor the simple event loop into a formal FSM. Define states for `PRE_TRADE`, `OPENING_AUCTION_CALL`, `CONTINUOUS_TRADING`, `CLOSING_AUCTION_CALL`, and `POST_CLOSE`.
2. **Implement Time-Based Transitions:** Create a simulated clock that drives transitions between states at the correct times (e.g., moving from `OPENING_AUCTION_CALL` to `CONTINUOUS_TRADING` at 08:00).
3. **Implement the Auction Uncrossing Algorithm:** Code the multi-stage auction logic as described in Section 2.3 (maximize volume, minimize surplus, etc.). This algorithm will be triggered by the FSM upon transitioning out of an auction state.
4. **Integrate State-Dependent Order Handling:** Modify the order processing logic to first query the FSM's current state. Based on the state, the system will correctly accept, reject, or "park" incoming orders as per the LSE rules.
5. **Parameterize Timings:** Refactor the hardcoded transition times to be loaded from a configuration file that mimics the *Business Parameters* document.

Successfully completing this phase is the most critical part of the project. It establishes the core architecture required for high-fidelity simulation.

4.3 Phase 3: Advanced Functionality and Realism

With the core state machine in place, this phase involves layering on the full suite of LSE features and nuances.

- **Objective:** Implement the complete set of LSE order types, auction extensions, and other microstructure rules.
- **Tasks:**

1. **Add All Complex Order Types:** Implement classes and handling logic for Iceberg, Hidden, Mid-Price Pegged, Stop, and Stop Limit orders. This requires carefully coding their unique behaviors, such as the pro-rata allocation for iceberg hidden volume and the parking/election logic for stop and pegged orders.
2. **Add All TIFs:** Implement the remaining Time-in-Force conditions, especially the auction-specific ones like ATC, GFA, and GFS, ensuring their interactions with the FSM are correct.
3. **Implement Auction Extensions:** Integrate the logic for Price Monitoring and Market Order extensions into the auction uncrossing process. This includes adding the random uncrossing delay.
4. **Implement Nuanced Rules:** Add logic for rules like Minimum Quantity at Touch (MQAT) and Large-in-Scale (LIS) validation for hidden orders.
5. **Full Parameterization:** Expand the configuration loader to manage all relevant parameters from the *Business Parameters* document, including price monitoring percentages, order value limits, and LIS thresholds.

4.4 Phase 4: Multi-Model Architecture and Data Integration

The final phase generalizes the simulator to handle the LSE's full spectrum of trading services and prepares it for use with real historical data.

- **Objective:** Support both SETS and SETSqx market models and integrate a pipeline for high-fidelity historical data.
- **Tasks:**
 1. **Refactor to a "Market Model" Architecture:** Generalize the FSM and rule engine into an abstract MarketModel base class.
 2. **Create Concrete Implementations:** Develop SETSModel and SETSqxModel classes that inherit from the base class. The SETSqxModel will have a different state machine, with multiple intra-day auction states and no continuous trading state.
 3. **Implement Dynamic Model Loading:** The simulator should be able to instantiate the correct market model for each security based on its reference data (e.g., loading a list of securities and their assigned trading service).
 4. **Develop a Data Ingestion Pipeline:** Build tools to parse and replay high-fidelity historical data. The ideal source would be the LSE's own historical data products, which provide tick-by-tick data. The parser must be able to convert this data into a sequence of events (new order, cancel, modify) that the simulator can process.

This phased roadmap provides a structured path to de-risk a highly complex project. It prioritizes the most difficult architectural challenges (the state machine) early in the process, ensuring the foundation is sound before adding the full feature set.

Conclusion and Strategic Recommendations

The endeavor to create a high-fidelity simulation of the London Stock Exchange in Python is a significant but achievable software engineering challenge. The analysis reveals that the core complexity lies not in the basic price-time priority matching of a continuous market, but in accurately modeling the LSE's comprehensive, multi-state, and multi-model operational environment. The intricate interplay between numerous order types, specific time-in-force

conditions, and the distinct rules of each trading phase—particularly the sophisticated auction mechanics—is the primary source of difficulty and the area where existing open-source tools are most deficient.

A review of the open-source landscape confirms that there is no turnkey solution. Available tools are either high-level strategy backtesters that abstract away crucial market mechanics, or more specialized limit order book implementations that lack the necessary state-aware architecture. Even advanced trading platforms like NautilusTrader are designed as clients to connect to an exchange, not to simulate the exchange itself. Therefore, a developer cannot simply adopt an existing project but must either build a new system or perform a substantial architectural refactoring of a foundational framework.

This leads to a critical strategic decision: **Build vs. Adapt.**

1. **Adapt an Existing Platform:** Leveraging a robust, event-driven framework like NautilusTrader offers the advantage of a production-grade infrastructure, including high-performance components and nanosecond-resolution timing. However, it would require the complex task of "inverting" its logic to create a venue simulator adapter that contains all the LSE's rules internally. This path leverages existing quality but introduces architectural complexity.
2. **Build from Scratch:** Following the phased roadmap outlined in this report, starting with a core limit order book and building the state machine and rule engine from the ground up, offers maximum control and ensures every component is purpose-built for the simulation. This approach is a larger initial effort but avoids the compromises and potential architectural friction of adapting a tool designed for a different purpose.

Final Recommendation:

For a project where the primary goal is maximum fidelity and a precise implementation of the LSE's official rulebook, the recommended path is to **build the simulator from scratch, adhering to the phased roadmap**. This approach, while more demanding, ensures that the architecture is tailored specifically to the problem domain without compromise.

The development effort should be disproportionately focused on **Phase 2: The Trading Day State Machine**. This is the most complex and critical component, and it is the area least addressed by the open-source community. By successfully implementing a robust, parameter-driven state machine that can manage the LSE's trading phases and execute its auction algorithms correctly, the developer will have overcome the project's greatest hurdle. The subsequent phases of adding advanced order functionality and multi-model support become more manageable engineering tasks built upon this solid foundation. This strategy offers the most direct and rigorous path to achieving the user's goal of creating a truly realistic and valuable LSE simulation.

Works cited

1. How to simulate stock prices with Python - PyQuant News,
<https://www.pyquantnews.com/the-pyquant-newsletter/how-to-simulate-stock-prices-with-python>
2. RaghavsScarletSplendour/MonteCarloSimStockPrices: This project explores the application of Monte Carlo simulation techniques to predict stock price movements over time. Utilizing Python and libraries such as NumPy and Matplotlib, it offers a hands-on approach to understanding the stochastic nature of financial markets and the practical application of statistical methods in finance. - GitHub,
<https://github.com/RaghavsScarletSplendour/MonteCarloSimStockPrices>
3. 1: An example of a trading screen for the SETS (Stock Exchange... - ResearchGate,

https://www.researchgate.net/figure/An-example-of-a-trading-screen-for-the-SETS-Stock-Exchange-Electronic-Trading-System_fig1_37710914 4. A Comparison of London and U.S. Stock Exchange Listing Requirements for Foreign Equity Securities - CORE, <https://core.ac.uk/download/pdf/62547766.pdf> 5. SETSsx and SEAQ | London Stock Exchange, <https://www.londonstockexchange.com/equities-trading/asset-classes/shares-trading/setsqx-and-seaq> 6. Can I trade on the London Stock Exchange AIM Segment? | FAQ Tool - Degiro, <https://www.degiro.ch/helpdesk/en/trading-possibilities/can-i-trade-london-stock-exchange-aim-segment> 7. Rules and regulations Equities trading resources - London Stock Exchange, <https://www.londonstockexchange.com/resources/equities-trading-resources> 8. Rules of the London Stock Exchange (LSE Rules) - Practical Law, [https://uk.practicallaw.thomsonreuters.com/9-500-3777?transitionType=Default&contextData=\(sc.Default\)](https://uk.practicallaw.thomsonreuters.com/9-500-3777?transitionType=Default&contextData=(sc.Default)) 9. rules-of-the-london-stock-exchange-effective-5-february-2024.pdf, <https://docs.londonstockexchange.com/sites/default/files/documents/rules-of-the-london-stock-exchange-effective-5-february-2024.pdf> 10. OFF ORDER BOOK TRADING RULES - London Stock Exchange, https://docs.londonstockexchange.com/sites/default/files/documents/n0119_attach1.pdf 11. MIT201 – Guide to the trading system – London Stock Exchange, https://docs.londonstockexchange.com/sites/default/files/documents/mit201-guide-to-the-trading-system-15_6_20240429.pdf 12. docs.londonstockexchange.com, <https://docs.londonstockexchange.com/sites/default/files/documents/draft-1.25.0-pisces-mit-and-te-parameters-version-9.2.xlsx> 13. Shares Trading | London Stock Exchange, <https://www.londonstockexchange.com/equities-trading/asset-classes/shares-trading> 14. SETS - Practical Law, <https://uk.practicallaw.thomsonreuters.com/Glossary/UKPracticalLaw/I250174a0e8db11e398db8b09b4f043e0> 15. STSM122020 - Financial markets: issue and trading of shares: electronic order-book trading of shares - HMRC internal manual - GOV.UK, <https://www.gov.uk/hmrc-internal-manuals/stamp-taxes-shares-manual/stsm122020> 16. STSM122010 - Financial markets: issue and trading of shares: order and quote driven markets - HMRC internal manual - GOV.UK, <https://www.gov.uk/hmrc-internal-manuals/stamp-taxes-shares-manual/stsm122010> 17. Sets Definition | What Does Sets Mean - IG UK, <https://www.ig.com/uk/glossary-trading-terms/sets-definition> 18. London Stock Exchange - Wikipedia, https://en.wikipedia.org/wiki/London_Stock_Exchange 19. FAQs | London Stock Exchange, <https://www.londonstockexchange.com/personal-investing/faqs> 20. Stock market hours - FOREX.com, <https://www.forex.com/en/news-and-analysis/stock-market-hours/> 21. London Stock Exchange Market Close ceremony, https://docs.londonstockexchange.com/sites/default/files/documents/market_close_information_document.pdf 22. LONDON STOCK EXCHANGE MARKET CLOSE CEREMONY, https://docs.londonstockexchange.com/sites/default/files/documents/market_close_information_document_1.pdf 23. Is the Stock Market Open Today? - Morningstar, <https://global.morningstar.com/en-gb/markets/is-stock-market-open-today> 24. LSE outage caused by matching engine logic issue - Global Trading, <https://www.globaltrading.net/lse-outage-caused-by-matching-engine-logic-issuue/> 25. How Order Matching Engines Process Trades: A Complete Guide ..., <https://www.quantvps.com/blog/order-matching-engines> 26. LSE SETS Hidden Order Enhancements - London Stock Exchange, https://docs.londonstockexchange.com/sites/default/files/documents/LSEG_CM_SETS_HIDDEN_ORDERS_BROCHURE_CMYK_LR.pdf 27. London Stock Exchange (LSE) - Real Trading,

<https://realtrading.com/trading-markets/lse/> 28. What is the Opening and Closing Auction on the LSE? - YouTube, https://www.youtube.com/watch?v=sV40TBikG_8 29. LSE | London Stock Exchange - POEMS, <https://www.poems.com.sg/markets/united-kingdom-lse/> 30. Trade and Invest in London Stock Exchange:, <https://www.utrade.com.my/pdf/faqs-london-market-on-utrade.pdf> 31. Python Trading Simulators to Test Strategies - PyQuant News, <https://www.pyquantnews.com/free-python-resources/python-trading-simulators-to-test-strategies> 32. Best Python Libraries for Algorithmic Trading and Financial Analysis - QuantInsti Blog, <https://blog.quantinsti.com/python-trading-library/> 33. Python Libraries for Quantitative Trading | QuantStart, <https://www.quantstart.com/articles/python-libraries-for-quantitative-trading/> 34. freqtrade/freqtrade: Free, open source crypto trading bot - GitHub, <https://github.com/freqtrade/freqtrade> 35. danielktaylor/PyLimitBook: Python implementation of fast ... - GitHub, <https://github.com/danielktaylor/PyLimitBook> 36. mikedh/lobby: Fully functioning fast Limit Order Book written in Python - GitHub, <https://github.com/mikedh/lobby> 37. ziaagikian/limit-order-book: Fast, Multi threaded and Efficient Trade Matching Engine, <https://github.com/ziaagikian/limit-order-book> 38. Surbeivol/PythonMatchingEngine: High performance ... - GitHub, <https://github.com/Surbeivol/PythonMatchingEngine> 39. silue-dev/limit-order-book-market-making: Market making ... - GitHub, <https://github.com/silue-dev/limit-order-book-market-making> 40. NautilusTrader: The fastest, most reliable open-source trading platform, <https://nautilustrader.io/> 41. nautechsystems/nautilus_trader: A high-performance ... - GitHub, https://github.com/nautechsystems/nautilus_trader 42. QuantConnect.com: Open Source Algorithmic Trading Platform., <https://www.quantconnect.com/> 43. LSE Market Data | Data Analytics - LSEG, <https://www.lseg.com/en/data-analytics/financial-data/pricing-and-market-data/equities-market-data/lse-market-data> 44. Technical Specification - London Stock Exchange, https://docs.londonstockexchange.com/sites/default/files/documents/HDP_PTD_Technical_Specification.pdf